Reproducing ReLA/GRES on gRefCOCO: failure-mode analysis and a post-hoc abstain-or-clarify detector

Visual Media (Eizo Media-gaku) — Final Report

Written in English by default; a Japanese version can be produced on request.

Every number and figure in this report is generated by the scripts in `shiinayane/visual-media-gres` (`scripts/`, `results/`) and regenerates from one inference pass per split.

1. Identity

Name: WANG Yankai **Student ID:** 48256454
Department: 情報理工学系研究科 電子情報学専攻
Laboratory: 鈴村研究室 (Suzumura Lab) **Research topic:** Vision–Language–Action / human–robot collaboration.

2. Target paper

GRES: Generalized Referring Expression Segmentation (Liu, Ding, Jiang, *CVPR 2023 Highlight*; network **ReLA**) generalizes classic Referring Expression Segmentation (RES). Classic RES assumes *exactly one* target per expression. GRES relaxes this to allow **multi-target** expressions ("the two bottles on the left") and **no-target** expressions, whose referent is *absent* from the image. To support and measure this, the authors release **gRefCOCO** (COCO train2014 images, RefCOCO-style expressions; train/val/testA/testB) with an evaluation protocol: **gIoU** (mean per-sample IoU; a correct no-target sample scores 1), **cIoU** (cumulative intersection / cumulative union), **N-acc** (no-target recall = TP/(TP+FN)), **T-acc** (target retention = TN/(TN+FP)) and **Pr@{0.7,0.8,0.9}**. ReLA is built on Mask2Former/Detectron2 and reaches state of the art on gRefCOCO.

3. Understanding of the method

Reading the released code (`gres_model/`), the inference path is:

1. A **backbone** (Swin-T/-B or R50) extracts multi-scale visual features and a **BERT-base** encoder embeds the expression; the backbone is language-aware (early fusion).
2. An **MSDeformAttn pixel decoder** builds a high-resolution mask feature map.
3. The **ReLA decoder** (`MultiScaleMaskedReferringDecoder`): 100 learnable *region queries* iterate through (i) Region–Image cross-attention, (ii) a first-layer Region–Language cross-attention gated by a learned `lang_weight`, and (iii) region–region self-attention. Two heads matter at test time: a *minimap/mask* branch yielding a 2-channel mask `pred_masks`$\in \mathbb{R}^{2\times H\times W}$ (`argmax` over channels is the binary output), and a *no-target (NT) head*—a small MLP applied per region and **averaged over all 100 regions** to a single 2-way NT logit.
4. **Inference** sigmoids both outputs; the evaluator compares `argmax` of each to GT.

One observation motivates the rest of the report: the model already exposes two scalars at inference—the no-target score σ(nt_logit) and the foreground mask confidence—exactly the quantities needed to decide whether to act, abstain, or ask. ReLA collapses each with a hard `argmax` at 0.5 and always commits to a mask or "empty." Section 6 keeps the trained model unchanged and replaces this hard decision with a calibrated abstain-or-clarify policy.

Table 1: Working software stack for the Blackwell/aarch64 + CUDA-13 server (single GPU, inference).

component	version	note
Python	3.11	venv
PyTorch	2.7.1+cu128	from the cu128 wheel index (the default PyPI aarch64 wheel is CPU-only)
torchvision	0.22.1	matched to torch 2.7.1
detectron2	0.6 (from source)	CPU-only C++ ops via `FORCE_CUDA=0` (avoids compiling CUDA kernels with the mismatched system nvcc 13.0)
numpy	1.26.4	pinned <2 for the pycocotools / detectron2 ABI
transformers	4.40.2	BERT-base text encoder

4. Execution environment

Hardware. NVIDIA **GB10** (Grace–Blackwell, **aarch64**), CUDA driver 580 / **CUDA 13.0**, compute capability **sm_121**, 20 CPU cores, 119 GB RAM. The stack the paper targets (torch 1.11 / CUDA 11.8 / detectron2 0.6, Python 3.7–3.9) does not run on a Blackwell/aarch64 machine, so a modern equivalent was assembled (single GPU, inference only; Table 1).

Two source patches (`patches/`, applied by `setup.sh`), both forced by the Blackwell + CUDA-13 mismatch:

1. **MSDeformAttn fallback.** The hand-written CUDA op cannot be compiled (system nvcc 13.0 vs torch cu128/12.8); I fall back to ReLA's own pure-PyTorch `ms_deform_attn_core_pytorch` (`F.grid_sample`), the reference implementation—numerically close, not bit-identical.
2. **sm_121 nvrtc fix.** `spatial_shapes.prod(1)` on int64 falls to the nvrtc jiterator, whose cu128 build rejects `-arch=compute_121`; I replace it with the identical elementwise product `shapes[:,0]*shapes[:,1]` (precompiled kernel).

Baseline reproduction (gRefCOCO val, Swin-T, 14,229 expressions): gIoU = 55.76, cIoU = 55.64, N-acc = 46.3%, T-acc = 99.9%, Pr@0.7 = 66.5. The paper reports Swin-T cIoU 57.73 / gIoU 56.86, so the result lands within ∼1–2 points—a faithful reproduction; the small gap is consistent with the `grid_sample` fallback and single-GPU evaluation and does not affect any conclusion below. Inference runs at ≈0.24 s/sample; one pass dumps compact per-sample signals to `results/*.pkl` and all downstream analysis is offline.

5. Failure case analysis

I study two failure conditions on gRefCOCO's official splits, so each claim is a measured rate, not an anecdote. F1 (no-target) is analysed on **val** (63% of its samples are no-target—the ideal stress test); F2 (multi-target) on **testA**, the split that contains a single-target baseline to compare against.

5.1 F1 — No-target hallucination (primary)

Setup. Restrict to the no-target subset of val (gt_nt=True, n=8,905, 63% of val). A *hallucination* is a no-target sample on which the model still emits a non-empty mask.

Result. The model hallucinates on **53.7%** of no-target samples (4,784/8,905); N-acc = 46.3%. The opposite error is almost absent: T-acc = 99.9% (only 7 of 5,324 present-target samples are wrongly called empty). The hallucinated masks are not small artefacts—median 7.2% of the image (mean 9.1%), mean foreground confidence 0.68: the model tends to be *wrong with high confidence.*

Mechanism. Figure 1 plots the model's own no-target score $s_{\mathrm{nt}} = \sigma(\mathrm{nt\_logit})_1$ for present

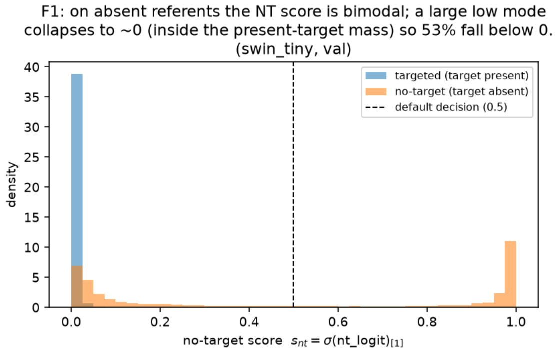

Figure 1: No-target score for present vs absent referents. On absent referents the score is bimodal; the large low mode collapses inside the present-target distribution, so 53% fall below 0.5 and a mask is committed.

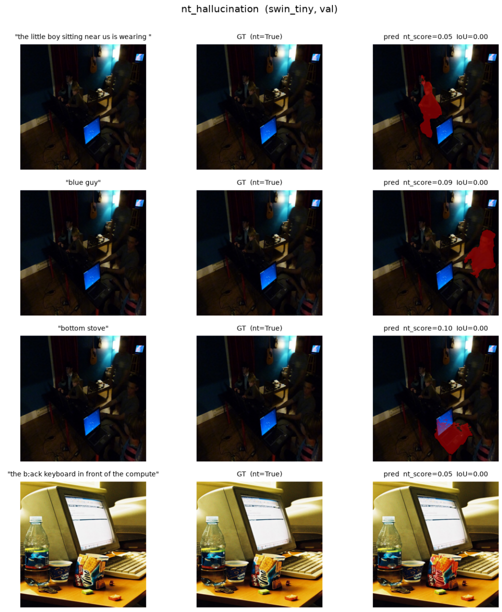

Figure 2: Each row: one *no-target* expression (left), the empty ground truth (middle), and the model's confident hallucinated mask in red (right).

vs absent referents. Present-target samples pile up at $s_{\mathrm{nt}} \approx 0$ (mean 0.012). Absent-referent samples are **bimodal**: a high mode (~37% above 0.9, correctly "no target") and a **low mode** (~38% collapsing to $s_{\mathrm{nt}} \approx 0$, inside the present distribution), so overall 53% fall below the 0.5 threshold (the empty-subset mean 0.48 is the average of two modes, not a typical value). Architecturally the NT head is a *single* scalar **averaged over all 100 region queries**; a plausible account is that a few regions confidently matching a salient distractor pull the mean below 0.5 and a mask is committed. I logged only the pooled s_{nt}, not per-region logits, so this is consistent with the architecture but not directly verified (Section 8). For a system that acts on language, this high-confidence false positive is the most costly error.

5.2 F2 — Multi-target under-segmentation

Setup. I use **testA** because (unlike val, whose targeted expressions are all multi-instance) it has a single-instance baseline (5,917 one-target, 5,940 two-target, 2,895 three-or-more). For each GT instance I record coverage $= |\mathrm{pred} \cap \mathrm{inst}|/|\mathrm{inst}|$, call it a *hit* if ≥ 0.5, and report per-instance recall.

Result (two effects). (i) Per-instance recall is **flat from one to two targets**—0.868 (single) vs 0.881 (two)—then **collapses to 0.594 at $\geq$3 targets**. (ii) Independently of count, coverage within a multi-target expression is **very uneven**: the best-covered instance averages 0.92 but the worst only 0.58, and the model **drops at least one instance entirely in 37.8%** of multi-target testA samples (26.1% on val). The under-segmentation is thus not "two worse than one on average"; it is a **capacity ceiling at $\geq$3 targets** plus **lopsided coverage** sacrificing the less salient instance.

Mechanism. ReLA predicts a single 2-channel "target" mask via an `einsum` over per-region minimap weights. One mask can represent one region, or two when the soft weighting is shared, but cannot spread mass across many disjoint instances—matching both the sharp drop at $\geq$3 and the dropped weaker instance within a pair.

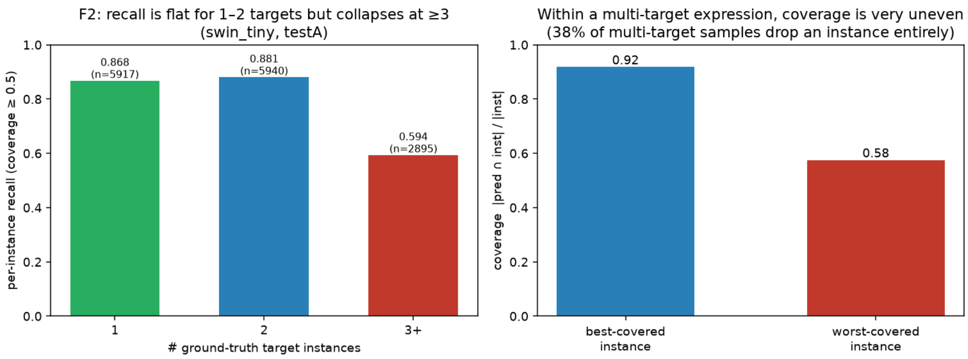

Figure 3: testA. *Left:* per-instance recall is flat for 1–2 targets (0.87, 0.88) but collapses at $\geq$3 (0.59). *Right:* within a multi-target expression the best instance is covered at 0.92 but the worst at only 0.58 (38% of samples drop one entirely).

Figure 4: Qualitative multi-target cases. Each row is one expression; ground-truth instances in green (middle), prediction in red (right). The model frequently captures the salient instance and drops the other.

6. Improvement: a post-hoc abstain-or-clarify detector

Motivated by my research on human–robot collaboration—where a system facing an unsatisfiable or ambiguous instruction should ask or abstain rather than commit—I keep the trained model unchanged (no retraining) and replace its hard `argmax` with a calibrated policy. For each sample I define $s_{\text{abstain}} = \max(s_{\text{nt}}, 1 - \overline{\text{mask\_conf}})$, high when the model leans "no target" *or* its committed mask is low-confidence. The policy is:

- $s_{\text{abstain}} \geq \tau \Rightarrow$ **ABSTAIN** (emit empty; scored as no-target);
- else if a mask has $\geq$2 connected components, the largest covering $< \rho$ of the area $\Rightarrow$ **CLARIFY** ("did you mean A or B?");
- else $\Rightarrow$ **COMMIT** the mask.

Table 2: gRefCOCO **val** (n=14,229). *In-sample*: τ is both chosen and scored here, so read this as the calibration curve, not a result.

op. point	gIoU	cIoU	N-acc	T-acc
baseline	55.76	55.64	46.3	99.9
safe 0.38	**66.9**	**58.2**	**65.0**	95.1
gIoU-opt 0.23	74.1	52.4	87.7	62.5

Table 3: gRefCOCO **testA** (held out, τ transferred unchanged; n=19,200). The honest evaluation.

op. point	gIoU	cIoU	N-acc	T-acc
baseline	65.03	65.42	50.6	99.0
safe 0.38	**65.9**	64.5	**63.3**	90.6
gIoU-opt 0.23	54.1	49.3	83.6	54.6

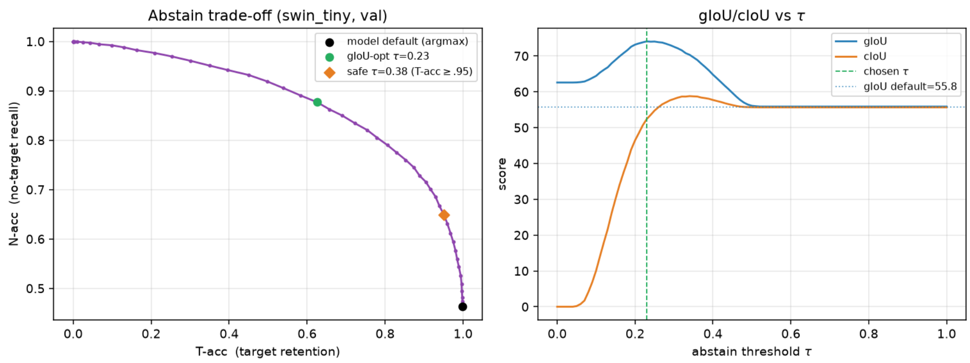

Figure 5: Abstain/clarify trade-off, calibrated on val. *Left:* N-acc vs T-acc as τ sweeps; the default `argmax` point (black) sits at the extreme no-abstention corner, the safe (orange) and gIoU-optimal (green) points move up the frontier. *Right:* gIoU and cIoU vs τ, both peaking above the default (dotted).

τ is calibrated on val (max gIoU) and applied unchanged to held-out testA. I also report a safety point τ_{safe}: the most aggressive abstention keeping T-acc ≥ 0.95. CLARIFY has no benchmark counterpart, so for scoring I keep the best-guess mask and report its trigger count separately. Calibration gives $\tau = 0.23$ (gIoU-optimal), $\tau_{\text{safe}} = 0.38$, $\rho = 0.7$.

7. Before/after comparison

τ is calibrated on **val** and applied **unchanged** to held-out **testA** (Tables 2–3; trade-off in Fig. 5).

On testA, with τ transferred unchanged, the detector is best described as a **safety re-weighting** rather than a segmentation-quality gain: the safe point raises N-acc by 12.7 points (50.6→63.3) at a cost of 8.4 points of T-acc (99.0→90.6), with overlap metrics essentially unchanged (gIoU +0.9, within the reproduction gap; cIoU −1.0). What it buys is a tunable, calibrated willingness to abstain on unsatisfiable instructions at a bounded retention cost. The aggressive gIoU-optimal τ *does not transfer*: tuned to val's 63%-no-target prior, on the 23%-no-target testA it over-abstains and lowers gIoU by 11 points. Hence only the T-acc-constrained point transfers safely, and a practical detector would calibrate τ to the deployment base-rate. The **CLARIFY** branch (separate from abstention) fired on 1,131 val / 1,712 testA committed predictions at τ=0.23—multi-component masks for which "which one?" is a reasonable alternative; an illustrative remedy for F2, reported as a trigger count since gRefCOCO has no clarification label.

8. Limitations

- Inference-only on the frozen Swin-T checkpoint (Swin-B downloaded but, per the resource guideline, not evaluated); with no retraining the detector reuses signals, not fixes the F2 bias.
- The `grid_sample` MSDeformAttn fallback is the reference op (close, not identical); the ∼1-point baseline gap is plausibly but not provably benign.

- The F1 mechanism is a hypothesis: I logged only the pooled s_{nt}, not per-region NT logits, so the averaging-dilution account is consistent with, but not proof of, the architecture.
- $\max(s_{\mathrm{nt}}, 1 - \mathrm{mask\_conf})$ is a simple interpretable rule with correlated terms; a learned calibrator over $\{s_{\mathrm{nt}}, \mathrm{mask\_conf}, \mathrm{area}, \#\mathrm{comp}\}$ would likely do better.
- On held-out data the detector is a safety re-weighting, not a quality gain, and a single global τ depends on the no-target base-rate. The CLARIFY trigger is a geometric heuristic reported only as a count (no clarification label; it does not yet read the expression).

9. Generative-AI usage

I used a generative-AI coding assistant (Anthropic's Claude, via the Claude Code CLI) throughout, and disclose it as follows. **The AI assisted with:** debugging the Blackwell/CUDA-13 environment and proposing the working stack and the two source patches (Section 4); writing the bulk of the analysis code in `scripts/` (inference loop, metric port, failure-mode and detector scripts, plotting); running the experiments; and drafting report text from the resulting numbers. **I am responsible for:** selecting the paper and the direction (the abstain-or-clarify framing tied to my HRC research); reviewing the code and setup; checking every reported number and figure against the generated tables; requesting the corrections from an internal review (e.g. re-centring F2 on testA; reporting held-out rather than in-sample detector results); and editing the report into final form. A detailed running log is in `GENAI_USAGE_LOG.md`.

10. Discussion

The two failure modes share an origin: ReLA makes a single hard decision at inference (one mask, one 0.5 no-target threshold), whereas the generalized task is partly about *uncertainty*—whether a referent exists at all, and how many. The no-target and multi-target subsets are exactly where committing to a single `argmax` is the wrong default. The detector adds no capacity; it reuses the uncertainty the model already encodes and turns it into three actions (act, abstain, ask), giving a calibrated, tunable abstention at a bounded retention cost that transfers to a held-out split. For a benchmark aimed at human–robot use, this argues for an explicit abstain/clarify option in how grounding models are both *scored* and *operated*, since these are the cases where a confident-but-wrong mask is most costly. The gains are modest and prior-dependent; natural next steps are a small learned calibrator over the four signals, an expression-aware CLARIFY trigger, and an abstention-aware metric (e.g. risk–coverage) reported alongside gIoU/cIoU.

Citation: Chang Liu, Henghui Ding, Xudong Jiang. "GRES: Generalized Referring Expression Segmentation." **CVPR 2023** (Highlight). arXiv:2306.00968.